Modul VII

Konsep Dasar Layout & Styling

A. Topik: Konsep dasar layout dan styling

B. Tujuan Pembelajaran

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

1. Menjelaskan konsep dasar layout pada Flutter serta perbedaan karakteristik Single-Child Widget dan Multi-Child Widget .
2. Menyusun hierarki komponen layout dalam bentuk Widget Tree yang terstruktur dan efisien.
3. Mengatur alignment dan posisi elemen pada Row dan Column menggunakan MainAxisAlignment dan CrossAxisAlignment.
4. Mengatur spacing dan jarak antar elemen UI menggunakan SizedBox dan Spacer.
5. Menerapkan widget Expanded dan Flexible dalam pembuatan layout.
6. Mengimplementasikan teknik layering UI menggunakan Stack dan Positioned.
7. Menerapkan styling dengan menggunakan Container dan BoxDecoration.

C. Alat dan Bahan

Untuk melaksanakan praktikum menggunakan *framework* Flutter, diperlukan perangkat dan perangkat lunak pendukung sebagai berikut:

C.1. Perangkat Keras (Hardware)

C.1.a Laptop/PC

- Prosesor: Minimal Intel Core i3 generasi ke-6 atau AMD Ryzen 3 setara (disarankan Intel Core i5/i7 atau Ryzen 5/7).
- RAM: Minimal 4 GB (disarankan 8 GB atau lebih untuk performa emulator yang lebih lancar).
- Penyimpanan: Minimal 10 GB ruang kosong (disarankan SSD 256 GB atau lebih).
- Kartu Grafis: Integrated GPU (Intel UHD/AMD Vega) sudah cukup, namun disarankan menggunakan dedicated GPU untuk mempercepat emulator.
- Layar: Resolusi minimal 1366 × 768 piksel (disarankan Full HD 1920 × 1080 piksel).
- Port USB/Type-C: Untuk menghubungkan perangkat Android fisik (opsional).

C.1.b Perangkat Tambahan

- Smartphone Android/iOS (opsional, untuk pengujian langsung di perangkat nyata).
- Kabel data USB (opsional, jika menggunakan perangkat fisik).

C.1.c Koneksi Internet Dibutuhkan untuk:

- Mengunduh Flutter SDK dan Android SDK.
- Mengunduh dependencies (package) saat pertama kali membuat proyek
- Flutter.
- Mengakses dokumentasi resmi Flutter.

C.2. Perangkat Lunak (Software)

C.2.a Sistem Operasi

- Windows: Windows 10/11 (64-bit).
- Linux: Ubuntu 20.04 LTS atau distro lain (64-bit).
- macOS: Versi Catalina (10.15) atau lebih baru.

C.2.b Flutter SDK (termasuk Dart SDK)

- Unduh dari: https://docs.flutter.dev/install
- Ekstrak dan tambahkan ke PATH sistem operasi.

C.2.c Android SDK dan Android Studio

- Android Studio (opsional, digunakan untuk emulator Android).
- Jika tidak menggunakan emulator, dapat memakai perangkat Android fisik.

C.3. d. Editor Kode

- Visual Studio Code (VS Code) dengan ekstensi Flutter dan Dart
- Alternatif: Android Studio dengan *plugin* Flutter & Dart.

C.4. e. Emulator/Simulator (opsional, bila tidak ada perangkat fisik)

- Android Emulator (via Android Studio).
- iOS Simulator (hanya untuk macOS dengan Xcode).

D. Dasar Teori

D.1. Widget Layout: Single-Child vs Multi-Child

Salah satu pilar utama arsitektur Flutter adalah filosofi desain "Everything is a Widget" yang dipadukan dengan prinsip rekayasa perangkat lunak modern: "Favor Composition over Inheritance" dimana lebih mengutamakan komposisi perakitan dibanding pewarisan kelas yang gemuk. Alih-alih membuat satu kelas widget raksasa dengan ratusan properti kompleks (seperti elemen pada HTML yang menampung styling, margin, animasi, dan layout sekaligus), Flutter memecah tanggung jawab tata letak ke dalam widget-widget kecil yang ramping, terfokus, dan dapat disusun layaknya balok Lego (composability).

Berdasarkan kemampuannya menampung elemen turunan di dalam Widget Tree, tata letak widget di Flutter diklasifikasikan secara tegas menjadi dua taksonomi besar:

1. **Single-Child Widgets**
   Single-child widget adalah komponen tata letak yang dirancang secara khusus untuk membungkus tepat satu widget turunan melalui parameter child: Widget. Peran utama kelompok ini bukanlah menata banyak data, melainkan memodifikasi karakteristik visual atau spasial dari satu elemen tunggal yang dibungkusnya.
2. **Multi-Child Widgets**
   Multi-child widget adalah komponen tata letak yang memiliki kemampuan menampung dan mengelola banyak elemen turunan sekaligus dalam format koleksi terurut (list/array) melalui parameter children: <Widget>[]. Kelompok widget ini bertindak sebagai manajer ruang yang bertugas mendistribusikan piksel layar kepada sekumpulan child secara berurutan.

Kategori	Karakteristik	Properti	Contoh
Single-Child	Hanya menerima tepat satu anak widget	child: Widget	Container, Padding, Center, Align, SizedBox, Expanded
Multi-Child	Menerima banyak anak widget dalam bentuk array/list	children: <Widget>[]	Row, Column, Stack, Wrap, Flex

D.2. Row & Column

Ketika mengamati anatomi antarmuka aplikasi bergerak, mulai dari aplikasi perpesanan instan, perbankan, transportasi daring, hingga dasbor pemantauan sensor IoT, hampir seluruh struktur visual yang tampak sesungguhnya dibangun dari perpaduan hierarkis tata letak linier (linear layout). Pola perancangan ini bukan kebetulan semata, melainkan konsekuensi logis dari bentuk fisik layar ponsel pintar yang berorientasi portrait serta pola alamiah mata manusia dalam mencerna informasi visual: membaca kata demi kata secara mendatar dari kiri ke kanan, lalu mengalirkan perhatian baris demi baris dari atas ke bawah. Keterampilan membedah sketsa desain (UI Slicing) menjadi baris dan kolom adalah keterampilan paling mendasar yang membedakan seorang pemula dari seorang mobile UI engineer profesional.

Untuk merealisasikan pola persepsi visual tersebut, Flutter menyediakan dua komponen tata letak linier paling fundamental: Row dan Column. Secara konseptual dan arsitektural, kedua widget ini bertindak sebagai *spatial layout managers* yang bertugas mendistribusikan sekumpulan elemen anak dalam satu dimensi arah aliran.

1. Widget Row menata anak-anaknya secara berurutan di sepanjang sumbu horizontal (mendatar dari kiri ke kanan). Contoh penerapannya dapat dilihat pada bagian judul kartu identitas warga (*header card*) yang meletakkan teks nama kota berdampingan dengan lambang daerah, baris informasi cuaca yang menampilkan ikon awan di samping angka suhu, atau baris tombol aksi yang menempatkan tombol konfirmasi tepat di sebelah tombol pembatalan.
2. Widget Column menata anak-anaknya secara berurutan di sepanjang sumbu vertikal (menurun dari atas ke bawah). Contoh penerapannya adalah struktur formulir aduan masyarakat yang menampilkan label judul aduan di baris paling atas, diikuti kotak isian uraian masalah di bagian tengah, indikator status penanganan di bawahnya, dan tombol kirim laporan di baris paling bawah.

Secara arsitektur internal di dalam Flutter framework, baik Row maupun Column sesungguhnya bukan kelas independen yang dibangun dari nol, melainkan merupakan turunan langsung dari base class yang sama, yaitu Flex. Perbedaan hakiki di antara keduanya murni terletak pada konfigurasi parameter direction. Row mengonfigurasi dirinya dengan parameter direction: Axis.horizontal, sedangkan Column mengonfigurasi dirinya dengan parameter direction: Axis.vertical. Keduanya mengimplementasikan layout negotiation protocol yang identik dalam membagi piksel ruang kosong dan mengukur dimensi fisik masing-masing komponen yang terdaftar di dalam list children.

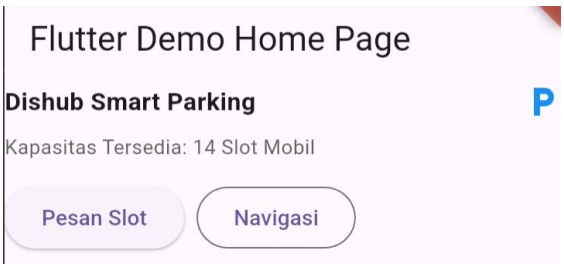
Salah satu karakteristik arsitektural paling krusial yang wajib dipahami oleh setiap pengembang adalah default sizing behavior dari kedua widget ini. Secara default, properti mainAxisSize pada Row dan Column bernilai MainAxisSize.max. Artinya, pada sumbu utamanya, kedua widget ini akan berperilaku sangat greedy dengan merenggang dan menghabiskan seluruh kapasitas bentangan layar yang tersedia, di mana Row akan melar memenuhi seluruh lebar layar gawai, sedangkan Column akan memanjang dari puncak layar teratas hingga dasar layar paling bawah meskipun konten di dalamnya hanya berupa dua baris teks pendek. Jika Anda sedang menyusun Card, AlertDialog, atau ModalBottomSheet, Anda wajib mengubah perilakunya menjadi MainAxisSize.min agar container tersebut shrink-to-fit pas membungkus total ukuran konten anak-anaknya saja, mencegah terjadinya ruang kosong mubazir yang merusak hierarki visual antarmuka.

Memahami properti konstruktor pada Row dan Column sangat penting agar antarmuka yang Anda bangun tidak hanya tampil rapi dan proporsional, tetapi juga berkinerja tinggi serta bebas dari bug.

1. children: <Widget>[]: Parameter wajib berupa list widget yang memuat seluruh komponen yang ingin Anda susun. Urutan penulisan kode di dalam kurung siku [] mencerminkan urutan render visual di layar (anak pertama ditulis di indeks 0 akan muncul di posisi paling awal).
2. mainAxisAlignment: Properti pengatur perataan pada Main Axis. Properti ini menentukan bagaimana free space yang belum terpakai akan didistribusikan kepada anak-anak widget.
3. crossAxisAlignment: Properti pengatur perataan pada Cross Axis. Menentukan bagaimana anak-anak widget yang memiliki tinggi atau lebar berbeda akan disejajarkan satu sama lain secara tegak lurus.
4. mainAxisSize: Menentukan seberapa rakus Row atau Column mengambil ruang pada sumbu utamanya:
   - MainAxisSize.max (default): Widget merenggang dan menghabiskan seluruh bentangan ruang sumbu utama yang tersedia.
   - MainAxisSize.min: Widget menyusut dan hanya mengambil ruang sebesar total akumulasi ukuran anak-anaknya saja.
5. verticalDirection: Menentukan arah pembacaan vertikal pada Column, apakah dari atas ke bawah (VerticalDirection.down, default) atau dibalik dari bawah ke atas (VerticalDirection.up).

Contoh Kode Penerapan Kombinasi Row & Column:

Berikut adalah contoh struktur nyata pembuatan panel informasi Smart Parking Card yang menggabungkan Column sebagai pembungkus vertikal dan Row sebagai pembungkus baris horizontal.



```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: Scaffold(
        appBar: AppBar(title: const Text('Flutter Demo Home Page')),
        body: Column(
          mainAxisSize: MainAxisSize.min,
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Row(
              mainAxisAlignment: MainAxisAlignment.spaceBetween,
              children: [
                Text(
                  'Dishub Smart Parking',
                  style: TextStyle(fontWeight: FontWeight.bold, fontSize: 16),
                ),
                Icon(Icons.local_parking, color: Colors.blue),
              ],
            ),
            SizedBox(height: 8),

            Text(
              'Kapasitas Tersedia: 14 Slot Mobil',
              style: TextStyle(color: Colors.grey[700], fontSize: 13),
            ),
            SizedBox(height: 12),

            Row(
              children: [
                ElevatedButton(onPressed: () {}, child: Text('Pesan Slot')),
                SizedBox(width: 8),
                OutlinedButton(onPressed: () {}, child: Text('Navigasi')),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```

Penjelasan Alur Kode Baris Demi Baris di Atas:

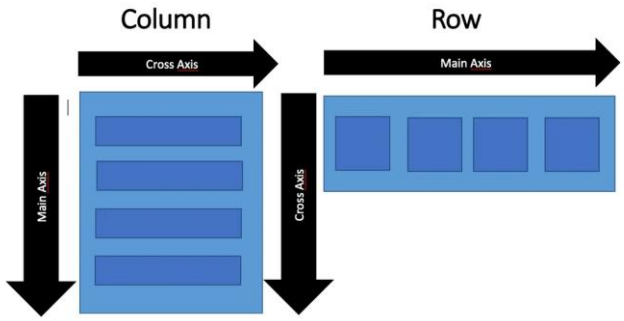
1. Column(...): Bertindak sebagai kerangka induk tata letak. Widget ini memastikan seluruh komponen (baris header, teks kapasitas, dan baris tombol aksi) tersusun rapi secara menurun dari atas ke bawah.
2. mainAxisSize: MainAxisSize.min: Membatasi bentangan vertikal Column agar tingginya pas membungkus konten di dalamnya. Tanpa baris ini, Column akan memanjang memenuhi seluruh tinggi layar dan mendorong elemen berikutnya keluar batas pandang.
3. crossAxisAlignment: CrossAxisAlignment.start: Mengatur garis batas perataan sumbu silang agar menempel rata di sisi kiri, sehingga teks kapasitas dan baris tombol tersusun sejajar lurus di sisi kiri kartu.
4. Row(mainAxisAlignment: MainAxisAlignment.spaceBetween, ...): Baris pertama menata teks judul dan ikon parkir secara mendatar. Pilihan MainAxisAlignment.spaceBetween

secara otomatis menghitung sisa lebar layar dan menempatkan teks di ujung kiri kartu serta ikon di ujung kanan kartu secara simetris.

5. SizedBox(height: 8) & SizedBox(height: 12): Berfungsi sebagai blok pembatas jarak vertikal statis yang memberikan ruang napas antar-baris tanpa perlu membungkus masing-masing teks dengan widget margin yang berulang.
6. Row(children: [ElevatedButton, SizedBox(width: 8), OutlinedButton]): Baris kedua menempatkan dua tombol aksi berdampingan secara horizontal. Penggunaan SizedBox(width: 8) di antara kedua tombol memberikan pemisah fisik sebesar 8 piksel agar tombol tidak saling bertabrakan.

D.3. Main Axis vs Cross Axis

Salah satu konsep yang paling sering membingungkan pengembang Flutter pemula adalah pemahaman mengenai Relative Axes. Di dalam Flutter, koordinat perataan tidak dinamai dengan istilah absolut seperti "horizontal" dan "vertikal", melainkan menggunakan istilah Main Axis dan Cross Axis. Alasan arsitektural di balik penamaan ini adalah karena arah sumbu tersebut bersifat dinamis dan bergantung sepenuhnya pada jenis widget linear yang sedang Anda gunakan.



- Pada widget Row, aliran elemen bergerak mendatar dari kiri ke kanan. Oleh karena itu, Main Axis pada Row adalah sumbu horizontal, sedangkan sumbu silang tegak lurusnya adalah Cross Axis pada Row yaitu sumbu vertikal.
- Sebaliknya, pada widget Column, aliran elemen bergerak menurun dari atas ke bawah. Oleh karena itu, Main Axis pada Column adalah sumbu vertikal, sedangkan sumbu silang tegak lurusnya adalah Cross Axis pada Column yaitu sumbu horizontal.

1. MainAxisAlignment

Konfigurasi MainAxisAlignment menentukan bagaimana Flutter mendistribusikan sisa free space pada sumbu utama setelah seluruh child selesai dihitung ukurannya.

a. MainAxisAlignment.start (default): Seluruh child dirapatkan menempel di awal sumbu utama (sisi paling kiri pada Row, atau puncak paling atas pada Column).
b. MainAxisAlignment.end: Seluruh child dirapatkan menempel di ujung akhir sumbu utama (sisi paling kanan pada Row, atau dasar paling bawah pada Column). Sangat berguna untuk baris tombol aksi konfirmasi dialog ("Batal" dan "Simpan").
c. MainAxisAlignment.center: Seluruh child dikumpulkan berjejer tepat di titik tengah bentangan sumbu utama, dengan ruang kosong dibagi rata di sisi awal dan sisi akhir.

d. MainAxisAlignment.spaceBetween: Ruang kosong ekstra dibagikan secara merata hanya di antara elemen-elemen anak. Child pertama menempel di batas tepi paling awal, sedangkan child terakhir menempel di batas tepi paling akhir. Nilai ini adalah pilihan standar industri untuk membuat custom AppBar atau baris header kartu profil.
e. MainAxisAlignment.spaceAround: Ruang kosong didistribusikan merata di depan, belakang, dan di antara elemen anak, dengan ketentuan bahwa ruang kosong di batas tepi luar bernilai tepat setengah dari ruang kosong yang memisahkan antar-anak.
f. MainAxisAlignment.spaceEvenly: Seluruh ruang kosong dibagi rata secara matematis identik ke seluruh celah, baik jarak antar-anak maupun jarak dari anak terluar ke batas tepi layar. Menghasilkan tata letak yang sangat presisi dan proporsional untuk deretan menu ikon dasbor.

2. CrossAxisAlignment

Konfigurasi CrossAxisAlignment mengatur perataan elemen sepanjang sumbu silang yang tegak lurus.

a. CrossAxisAlignment.start: Meratakan seluruh child pada garis awal sumbu silang (sisi atas pada Row, atau sisi kiri pada Column). Sangat penting digunakan pada Column form agar teks label dan kotak isian tidak bergeser ke tengah.
b. CrossAxisAlignment.center (default): Menjajarkan titik tengah seluruh child pada sumbu silang. Pada Row, jika Anda memiliki ikon setinggi 40 piksel dan teks setinggi 16 piksel, teks akan otomatis berada di tengah-tengah ketinggian ikon secara simetris.
c. CrossAxisAlignment.end: Meratakan child pada ujung akhir sumbu silang (sisi bawah pada Row, atau sisi kanan pada Column).
d. CrossAxisAlignment.stretch: Memaksa seluruh child merenggang selebar atau setinggi kapasitas maksimum sumbu silang yang tersedia. Jika diterapkan pada Column, seluruh tombol anak di dalamnya akan otomatis melebar selebar layar penuh tanpa perlu membungkus masing-masing tombol dengan SizedBox(width: double.infinity).
e. CrossAxisAlignment.baseline: Menjajarkan elemen teks yang memiliki ukuran font berbeda agar bertumpu pada alphabetic baseline yang sama. Jika Anda menggunakan opsi ini, Flutter mewajibkan pendefinisian parameter pendamping: textBaseline: TextBaseline.alphabetic.

D.4. Pengaturan Spacing: SizedBox vs Spacer

Ketika menyusun antarmuka aplikasi, salah satu kebiasaan buruk yang kerap dilakukan oleh pengembang pemula adalah menyalahgunakan widget Padding atau Container bersarang hanya untuk memisahkan dua buah widget sejauh beberapa piksel. Pola penulisan tersebut memicu masalah Widget Tree Bloat serta membuat kode program sulit dibaca (nesting hell).

Flutter menyediakan dua pendekatan elegan, bersih, dan berkinerja tinggi untuk mengatur jarak antar-komponen:

1. SizedBox (Fixed Spacing)

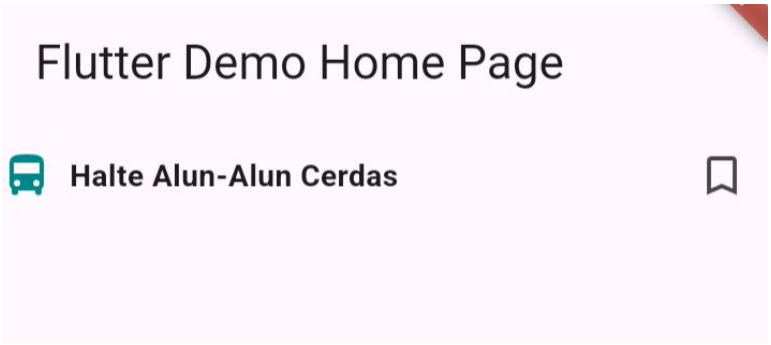
SizedBox adalah widget single-child yang memiliki dimensi lebar (width) dan tinggi (height) bernilai pasti dalam satuan dp. Ketika digunakan tanpa widget child, SizedBox berfungsi layaknya balok ruang transparan yang mendorong elemen di sebelahnya:

- SizedBox(width: 12): Menghasilkan balok pemisah horizontal tetap sebesar 12 piksel di dalam Row.
- SizedBox(height: 16): Menghasilkan balok pemisah vertikal tetap sebesar 16 piksel di dalam Column.
- Named Constructors:
  - SizedBox.shrink(): Menghasilkan kotak dengan ukuran sekecil mungkin (lebar 0, tinggi 0). Sering digunakan untuk menyembunyikan widget secara kondisional tanpa memakan ruang.
  - SizedBox.expand(): Memaksa widget anak memenuhi seluruh kapasitas ruang yang diberikan oleh parent-nya.
  - SizedBox.square(dimension: 48): Jalan pintas untuk membuat kotak bujur sangkar simetris berdimensi 48x48 piksel.

2. Spacer (Dynamic Spring)

Berbeda dengan SizedBox yang ukurannya kaku bernilai angka tetap, Spacer adalah widget non-visual yang dirancang untuk bertindak layaknya pegas dinamis. Di balik layar, widget Spacer sebenarnya dibangun dari komponen Expanded(child: SizedBox.shrink()). Karakteristik utamanya adalah akan mengambil seluruh sisa ruang kosong yang belum terpakai pada sumbu utama, sehingga mendorong widget yang berada di belakangnya menepi sejauh mungkin ke ujung yang berlawanan.

Contoh Kode Perbandingan SizedBox vs Spacer:




```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: Scaffold(
        appBar: AppBar(title: const Text('Flutter Demo Home Page')),
        body: Row(
          children: [
```

```
            Icon(Icons.directions_bus, color: Colors.teal),
            SizedBox(width: 8),
            Text(
              'Halte Alun-Alun Cerdas',
              style: TextStyle(fontWeight: FontWeight.bold),
            ),
            Spacer(),
            IconButton(icon: Icon(Icons.bookmark_border), onPressed: () {}),
          ],
        ),
      ),
    );
  }
}
```

Penjelasan Alur Kode Baris Demi Baris di Atas:

1. Icon(...) & Text(...): Diletakkan berurutan di sisi awal Row.
2. SizedBox(width: 8): Menjamin ada jeda napas tepat 8 piksel antara gambar ikon bus dan teks nama halte, mencegah teks menabrak ikon.
3. Spacer(): Mengukur sisa lebar layar gawai yang belum terpakai antara teks nama halte dan tombol bookmark. Widget ini memuai secara dinamis memenuhi celah tersebut, secara otomatis menolak tombol bookmark ke tepi kanan terluar tanpa memerlukan kalkulasi lebar layar manual menggunakan MediaQuery.
4. IconButton(...): Terdorong ke ujung kanan dan tetap berada di posisinya secara adaptif, baik saat aplikasi dibuka di smartphone layar sempit maupun tablet layar lebar.

D.5. Expanded dan Flexible

Salah satu momen paling umum yang dialami pengembang saat membangun antarmuka Flutter adalah kemunculan garis bergaris kuning hitam (hazard strip) disertai pesan error di konsol debug:

"A RenderFlex overflowed by xxx pixels on the right"

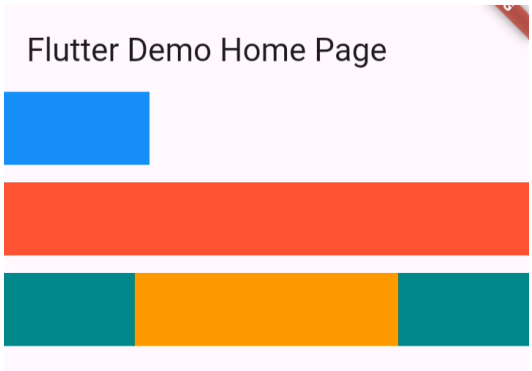
Penyebab mendasar terjadinya overflow ini bersumber langsung dari arsitektur tata letak Flutter:

1. Secara default, widget Row memberikan unbounded constraints (lebar maksimum bernilai tak terhingga) kepada seluruh child di dalamnya.
2. Ketika Anda meletakkan widget Text yang memuat kalimat panjang di dalam Row, widget Text akan mengukur dirinya berdasarkan panjang kalimat aslinya. Karena parent tidak memberikan batas lebar pasti, teks akan terus memanjang ke kanan dalam satu baris lurus tanpa pernah melakukan word-wrap.
3. Begitu akumulasi lebar elemen melebihi kapasitas piksel layar fisik ponsel (misalnya total teks butuh 500 piksel padahal layar hanya selebar 392 piksel), Flutter Engine menolak merender elemen yang keluar batas layar dan menandainya sebagai RenderFlex overflow error.

Flutter sengaja menampilkan error visual ini dan tidak melakukan silent clipping agar pengembang sadar bahwa tata letak antarmukanya berpotensi rusak jika dibuka di gawai pengguna yang memiliki resolusi layar lebih kecil. Untuk mengendalikan elemen yang berisiko meluap dan membagi ruang sisa secara proporsional, Flutter menyediakan dua widget pembungkus utama yaitu Expanded dan Flexible.

1. Widget Expanded
   Widget Expanded adalah komponen single-child yang memaksa widget anak di dalamnya untuk menghabiskan seluruh sisa ruang kosong yang tersedia pada Main Axis. Di balik layar, Expanded mengonfigurasi dirinya dengan aturan pengikatan ketat (fit: FlexFit.tight). Karakteristik ini secara otomatis memotong sifat unbounded constraints dari Row atau Column, sehingga memaksa widget anak (seperti Text atau Column) menerima batasan lebar pasti. Dengan adanya batasan lebar pasti tersebut, teks panjang di dalamnya dapat menghitung titik potong baris dan melakukan word-wrap secara rapi ke baris berikutnya.
2. Widget Flexible
   Berbeda dengan Expanded yang bersifat rakus dan mewajibkan anak menghabiskan seluruh ruang sisa, widget Flexible secara default beroperasi dengan aturan pengikatan longgar (fit: FlexFit.loose). Karakteristik ini memberikan kebebasan kepada widget anak untuk mempertahankan ukuran fisiknya sendiri jika kontennya berukuran kecil, namun tetap membatasi ukuran maksimumnya agar tidak melebihi sisa ruang layar jika kontennya membesar. Widget Flexible sangat ideal digunakan pada tombol atau badge informasi yang ingin tetap adaptif mengecil di layar sempit tanpa harus melar berlebihan di layar lebar.
3. Parameter Pembagian Proporsional (flex)
   Baik Expanded maupun Flexible dibekali properti integer bernama flex dengan nilai default 1. Properti ini bekerja berdasarkan rasio matematika pembagian ruang. Jika terdapat dua widget fleksibel berdampingan dengan konfigurasi flex: 2 dan flex: 1, Flutter akan membagi total sisa ruang menjadi 3 bagian: elemen pertama memperoleh alokasi (66,6%), sedangkan elemen kedua memperoleh alokasi (33,3%) secara dinamis dan responsif terhadap perubahan resolusi layar.

Contoh 1: Pembagian Layout




```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
```

```
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: Scaffold(
        appBar: AppBar(title: const Text('Flutter Demo Home Page')),
        body: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Row(
              children: [
                Flexible(
                  child: Container(height: 50, width: 100, color: Colors.blue),
                ),
              ],
            ),
            SizedBox(height: 12),
            Row(
              children: [
                Expanded(child: Container(height: 50, color: Colors.red)),
              ],
            ),
            SizedBox(height: 12),
            Row(
              children: [
                Expanded(
                  flex: 1,
                  child: Container(height: 50, color: Colors.teal),
                ),
                Expanded(
                  flex: 2,
                  child: Container(height: 50, color: Colors.orange),
                ),
                Expanded(
                  flex: 1,
                  child: Container(height: 50, color: Colors.teal),
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```

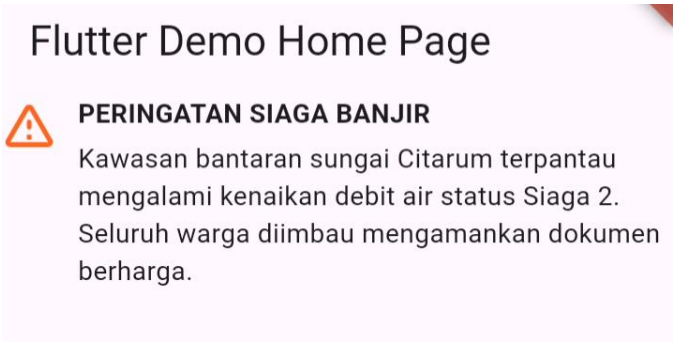
Penjelasan Alur Kode Baris Demi Baris di Atas:

1. Column(...): Bertindak sebagai kerangka induk vertikal yang menumpuk tiga baris eksperimen visual dari atas ke bawah.
2. Bagian 1: Meskipun berada di dalam Row, container biru dibungkus oleh Flexible. Karakteristik loose constraints (fit: FlexFit.loose) mengizinkan container mempertahankan ukuran aslinya yaitu tepat selebar 100 piksel (width: 100), membiarkan sisa ruang horizontal layar di sebelah kanannya tetap kosong tanpa dipaksa melar.
3. Bagian 2 container merah dibungkus oleh Expanded. Karakteristik tight constraints (fit: FlexFit.tight) memaksa container merah menghabiskan seluruh sisa ruang horizontal yang

tersedia, sehingga melar penuh memenuhi lebar layar smartphone dari ujung kiri ke ujung kanan.

4. Bagian 3: Tiga container disusun berdampingan dengan konfigurasi flex: 1 (Teal), flex: 2 (Oranye), dan flex: 1 (Teal). Flutter Engine menjumlahkan total menjadi 4 bagian, lalu membagi lebar layar secara proporsional.

Contoh 2: Penerapan Expanded Mengatasi Overflow Teks Panjang.




```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: Scaffold(
        appBar: AppBar(title: const Text('Flutter Demo Home Page')),
        body: Row(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Icon(
              Icons.warning_amber_rounded,
              color: Colors.deepOrange,
              size: 32,
            ),
            SizedBox(width: 10),
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    'PERINGATAN SIAGA BANJIR',
                    style: TextStyle(fontWeight: FontWeight.bold),
                  ),
                  SizedBox(height: 4),
                  Text(
                    'Kawasan bantaran sungai Citarum terpantau mengalami kenaikan
debit air status Siaga 2. Seluruh warga diimbau mengamankan dokumen berharga.',
                  ),
                ],
              ),
            ),
          ],
        ),
      ),
```

```
    );
  }
}
```

Penjelasan Alur Kode Baris Demi Baris di Atas:

1. Row(...): Menata ikon peringatan dan blok teks informasi secara mendatar dari kiri ke kanan.
2. Icon(...) & SizedBox(width: 10): Menampilkan simbol bahaya berdimensi tetap 32 piksel dan memberikan pemisah jarak statis 10 piksel dari teks di kanannya.
3. Expanded(...): Membungkus seluruh komponen Column teks. Widget ini menghitung sisa lebar horizontal layar yang tersisa setelah dikurangi lebar ikon dan jarak SizedBox, lalu memaksakan ukuran batas lebar tersebut ke dalam Column.
4. Column(...): Menerima batasan lebar pasti dari Expanded, menyusun judul siaga dan teks narasi secara vertikal dari atas ke bawah.
5. Text(...): Karena kini berada di dalam wadah yang lebarnya terbatas, teks narasi yang panjang secara otomatis memecah kalimatnya menjadi beberapa baris (word-wrap) tanpa meluber keluar dari batas layar ponsel, sehingga layout sepenuhnya bebas dari error RenderFlex overflow.

Tabel perbandingan Expanded vs Flexible:

Kriteria Komparasi	**Widget Expanded**	**Widget Flexible**
Sifat Alokasi Ruang	Wajib habis memenuhi seluruh ruang sisa yang tersedia pada Main Axis.	Boleh kurang dari total ruang sisa yang tersedia.
Perilaku pada Child Berukuran Kecil	Child dipaksa melar memenuhi seluruh porsi jatah ruangnya.	Child tetap mempertahankan ukuran aslinya tanpa dipaksa melebar.
Perilaku pada Child Berukuran Terlalu Besar	Child dipersempit agar tidak melebihi sisa ruang (mencegah overflow).	Child dipersempit agar pas di dalam sisa ruang (mencegah overflow).

D.6. Stack dan Positioned

Tata letak linear berbasis Row dan Column hanya mampu menyusun elemen secara sejajar pada bidang datar 2 dimensi (sumbu X horizontal atau sumbu Y vertikal). Namun, modern mobile design patterns sangat sering menuntut penataan elemen yang saling bertumpukan di atas elemen lain pada z-index. Skenario nyata di dunia industri yang membutuhkan penumpukan layer antara lain:

1. Menaruh lingkaran hijau penanda status online tepat di sudut bawah foto profil pengguna.
2. Menampilkan badge teks bertuliskan "live" atau "recorded" yang mengambang di pojok kanan atas sebuah banner foto berita bencana.
3. Menempatkan Floating Action Button atau kartu ringkasan rute tepat di atas peta navigasi digital.

Untuk memenuhi kebutuhan desain tersebut, Flutter menyediakan widget Stack dan Positioned.

D.6.a Aturan Render & Mekanika Perilaku Stack

- **Urutan Layering LIFO:**
  Aturan mutlak pada Stack adalah: elemen yang ditulis paling awal (indeks 0) pada list children akan digambar di layer dasar. Setiap elemen berikutnya akan digambar menimpa lapisan sebelumnya. Elemen yang dideklarasikan paling akhir pada list children akan muncul di layer teratas layar.
- **Properti alignment:**
  Menentukan titik gravitasi penempatan bawaan bagi anak-anak widget yang tidak memiliki koordinat eksplisit (misal: Alignment.center untuk meletakkan teks tepat di tengah-tengah gambar latar belakang).
- **Properti clipBehavior:**
  Mengatur perilaku pemotongan visual jika ada elemen anak yang ukurannya meluber keluar dari batas kotak Stack:
  - Clip.hardEdge: Memotong tepi elemen secara tajam agar tidak melewati batas kotak.
  - Clip.none: Memperbolehkan elemen anak (seperti badge status yang sengaja ditaruh setengah keluar di sudut kartu) tampak mengambang secara bebas tanpa clipping.

D.6.b Penempatan Presisi Menggunakan Positioned

Widget Positioned adalah widget single-child istimewa yang hanya sah digunakan sebagai anak langsung dari Stack. Jika Anda meletakkan Positioned di luar Stack, Flutter akan melempar pesan error. Widget ini bertindak layaknya jangkar penentu koordinat absolut:

- top: Jarak pasti elemen dari batas tepi atas parent Stack.
- bottom: Jarak pasti elemen dari batas tepi bawah parent Stack.
- left: Jarak pasti elemen dari batas tepi kiri parent Stack.
- right: Jarak pasti elemen dari batas tepi kanan parent Stack.
- width & height: Memaksakan dimensi fisik elemen secara langsung mengabaikan ukuran bawaan child aslinya.

Contoh program:

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: Scaffold(
        appBar: AppBar(title: const Text('Flutter Demo Home Page')),
        body: Center(
          child: SizedBox(
            width: 90,
            height: 90,
            child: Stack(
              clipBehavior: Clip.none,
              children: [
                CircleAvatar(
                  radius: 40,
```

```
                backgroundColor: Colors.blueGrey[100],
                child: Icon(
                  Icons.person,
                  size: 45,
                  color: Colors.blueGrey[700],
                ),
              ),
              Positioned(
                bottom: 2,
                right: 2,
                child: Container(
                  padding: EdgeInsets.symmetric(horizontal: 6, vertical: 2),
                  decoration: BoxDecoration(
                    color: Colors.green[600],
                    borderRadius: BorderRadius.circular(10),
                    border: Border.all(color: Colors.white, width: 2),
                  ),
                  child: Text(
                    'AKTIF',
                    style: TextStyle(
                      color: Colors.white,
                      fontSize: 9,
                      fontWeight: FontWeight.bold,
                    ),
                  ),
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

D.7. Container

Di antara ratusan widget yang disediakan oleh Flutter SDK, widget Container sering kali dijuluki sebagai Swiss Army Knife tata letak Flutter. Julukan ini disematkan karena Container adalah widget komposit single-child yang menggabungkan berbagai kapabilitas spasial dasar sekaligus ke dalam satu komponen praktis: penentuan ukuran pasti (width dan height), perataan posisi internal (alignment), pembatas jarak luar (margin), dan pembatas ruang dalam (padding).

Jika ditinjau dari arsitektur internal Flutter Engine, Container sesungguhnya bukanlah widget primitif yang merender piksel secara langsung, melainkan sebuah widget kenyamanan (convenience widget) yang secara cerdas membungkus widget-widget tata letak lain seperti LimitedBox, ConstrainedBox, Align, Padding, dan DecoratedBox. Jika Anda tidak mendefinisikan ukuran width dan height, Container secara default akan menyesuaikan ukurannya dengan ukuran widget anak di dalamnya (shrink-to-fit), atau mencoba melar sebesar mungkin memenuhi parent jika parent memberikan batasan ruang longgar. Di bawah ini adalah ilustrasi bagaimana sebuah container mengelola ruang di layar.

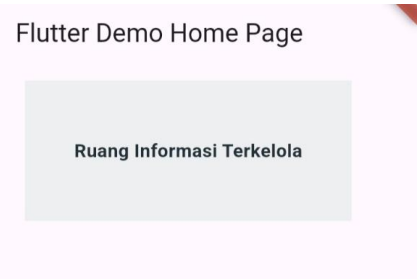


1. Content (Child Widget): Komponen inti yang berada di tengah-tengah (misalnya teks informasi, gambar, atau baris ikon).
2. Padding: Ruang napas pelindung di sisi dalam container yang memisahkan konten dari border. Tanpa padding, teks akan menempel sesak menabrak garis tepi.
3. Border & BorderRadius: Garis pembatas fisik container beserta kelengkungan pada keempat sudutnya.
4. Margin: Ruang kosong di sisi luar container yang menjaga jarak aman agar container tidak bertabrakan dengan widget-widget tetangga di sekelilingnya.

Nilai margin dan padding diatur menggunakan objek kelas EdgeInsets melalui empat metode konstruktor utama:

- EdgeInsets.all(16.0): Menerapkan jarak 16 piksel secara seragam ke keempat sisi sekaligus (atas, bawah, kiri, kanan).
- EdgeInsets.symmetric(horizontal: 16.0, vertical: 8.0): Membedakan jarak simetris: 16 piksel untuk sisi mendatar (kiri dan kanan), serta 8 piksel untuk sisi tegak (atas dan bawah).
- EdgeInsets.only(top: 12.0, left: 8.0): Memberikan jarak khusus hanya pada sisi-sisi tertentu yang didefinisikan secara eksplisit.
- EdgeInsets.fromLTRB(10, 20, 10, 20): Menentukan nilai piksel secara berurutan untuk sisi Left, Top, Right, dan Bottom.

Contoh:




```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
```

```
      theme: ThemeData(primarySwatch: Colors.blue),
      home: Scaffold(
        appBar: AppBar(title: const Text('Flutter Demo Home Page')),
        body: Container(
          width: 280,
          height: 120,
          margin: EdgeInsets.symmetric(horizontal: 24.0, vertical: 12.0),
          padding: EdgeInsets.all(16.0),
          alignment: Alignment.center,
          color: Colors.blueGrey[50],
          child: Text(
            'Ruang Informasi Terkelola',
            style: TextStyle(
              fontSize: 16,
              fontWeight: FontWeight.bold,
              color: Colors.blueGrey[900],
            ),
          ),
        ),
      ),
    );
  }
}
```

Penjelasan Alur Kode Baris Demi Baris di Atas:

- width: 280 & height: 120: Mengunci dimensi fisik container secara presisi berukuran lebar 280 piksel dan tinggi 120 piksel logis.
- margin: EdgeInsets.symmetric(...): Menjaga jarak luar sejauh 24 piksel di sisi kiri-kanan dan 12 piksel di sisi atas-bawah agar tidak berhimpitan dengan widget lain di sekitarnya.
- padding: EdgeInsets.all(16.0): Memberikan ruang napas dalam seragam sebesar 16 piksel di sekeliling teks agar teks tidak menabrak batas tepi kotak.
- alignment: Alignment.center: Memposisikan teks judul tepat di titik tengah geometris container.
- color: Colors.blueGrey[50]: Memberikan warna latar belakang dasar solid abu-abu kebiruan lembut tanpa memerlukan konfigurasi dekorasi kompleks.

D.8. BoxDecoration

Jika Container berperan sebagai pengatur kerangka spasial dan dimensi fisik, maka kelas BoxDecoration bertindak sebagai perancang busana estetika visualnya. Di dalam Flutter, dekorasi grafis sengaja dipisahkan ke dalam kelas khusus ini untuk memberikan keleluasaan dalam menciptakan kartu antarmuka bergaya modern, mulai dari kelengkungan sudut halus, pewarnaan gradasi dinamis, hingga bayangan realistis yang memberi ilusi kedalaman ruang tiga dimensi pada layar hanphone.

1. Color

Properti color digunakan untuk menentukan warna latar belakang solid container. Nilai warna dapat menggunakan palet bawaan seperti Colors.white, transparansi seperti Colors.black.withOpacity(0.1), atau kode heksadesimal 32-bit Color(0xFF1E3C72).

2. BorderRadius

Properti borderRadius mengubah sudut kotak yang kaku menjadi melengkung modern:

- BorderRadius.circular(16.0): Membulatkan keempat sudut secara seragam dengan radius kelengkungan 16 piksel.
- BorderRadius.only(...): Mengatur lengkungan secara spesifik pada sudut tertentu (misalnya hanya sudut kiri atas dan kanan bawah).

3. Border

Properti border menambahkan garis fisik penegas di sekeliling container untuk membedakan kartu dari latar belakang layar. Konfigurasi umum menggunakan Border.all(color: Colors.blueAccent, width: 1.5).

4. LinearGradient

Properti gradient menghasilkan transisi warna yang halus dari satu titik ke titik lain:

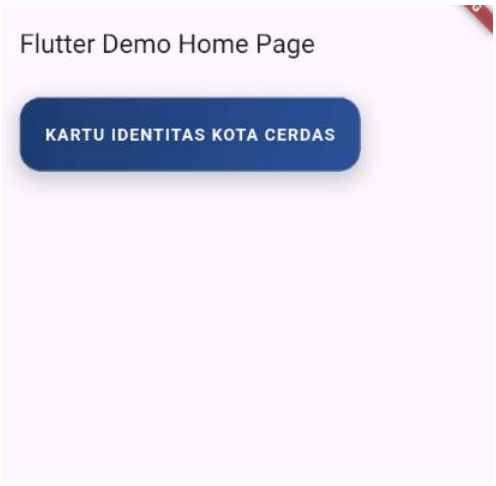
- begin: Koordinat awal transisi warna (misal: Alignment.topLeft).
- end: Koordinat akhir transisi warna (misal: Alignment.bottomRight).
- colors: Daftar warna yang dipadukan secara berurutan, misalnya [Color(0xFF1E3C72), Color(0xFF2A5298)].

5. BoxShadow

Properti boxShadow menerima list objek BoxShadow untuk memproyeksikan bayangan lembut yang memberikan ilusi fisik bahwa kartu mengambang di atas permukaan layar:

- color: Warna bayangan, disarankan menggunakan transparansi lembut Colors.black.withOpacity(0.15).
- blurRadius: Tingkat kelembutan bauran bayangan.
- spreadRadius: Jangkauan jarak pelebaran bayangan dari batas container.
- offset: Arah jatuhnya bayangan pada sumbu X dan Y, misalnya Offset(0, 6) untuk bayangan yang jatuh ke bawah.

Contoh program:



Penjelasan Alur Kode Baris Demi Baris di Atas:

1. margin: EdgeInsets.all(16.0): Menjaga jarak luar container sebesar 16 piksel dari batas layar device dan widget lain di sekitarnya.
2. padding: EdgeInsets.all(20.0): Memberikan ruang napas dalam sebesar 20 piksel agar teks judul di dalamnya tidak menabrak batas kartu, menciptakan ruang baca yang nyaman bagi mata pengguna.
3. borderRadius: BorderRadius.circular(18.0): Memotong sudut kotak yang kaku menjadi sudut melengkung modern sebesar 18 piksel logis.
4. gradient: LinearGradient(...): Memadukan warna biru laut tua heksadesimal 0xFF1E3C72 di sudut kiri atas ke warna biru cerah 0xFF2A5298 di sudut kanan bawah, memberikan kesan antarmuka berstandar premium.
5. boxShadow: [BoxShadow(...)]: Memproyeksikan bayangan lembut dengan keburaman 14 piksel yang bergeser 6 piksel ke bawah (Offset(0, 6)), memberikan ilusi fisik bahwa kartu sedang melayang di atas permukaan layar ponsel (elevation effect).
6. border: Border.all(color: Colors.white24, width: 1): Menambahkan garis batas tepi tipis berwarna putih semi-transparan (opasitas 24%) untuk memberikan aksen kontur visual yang tajam (subtle border).

E. Tugas Praktikum

Instruksi Umum Pengerjaan:

1. Ketikkan kode pada masing-masing kasus di bawah ini ke dalam editor Flutter Anda.
2. Lengkapi bagian yang rumpang (_____) sesuai dengan petunjuk pada komentar bernomor.
3. Jalankan program dan pastikan tidak ada *error*.
4. Buat kesimpulan singkat disetiap akhir kasus.
5. Kumpulkan seluruh file hasil pengerjaan Anda di akhir sesi praktikum ke dalam LMS.

Contoh:

Aplikasi membutuhkan program yang dapat menyapa pengguna secara otomatis dan mencetak umur mereka. Anda diminta melengkapi tipe data dan perintah cetak layar.

```
1   void main() {
2     // TODO 1: Lengkapi tipe data yang tepat untuk teks dan angka bulat
3     ____ nama = "Budi";
4     ____ umur = 20;
5
6     // TODO 2: Lengkapi perintah bawaan Dart untuk mencetak ke layar
7     ____("Halo, nama saya $nama dan umur saya $umur");
8
9     // Kesimpulan pada soal ini adalah:
10    // _____________________________________________________________
11  }
```

Jawaban:

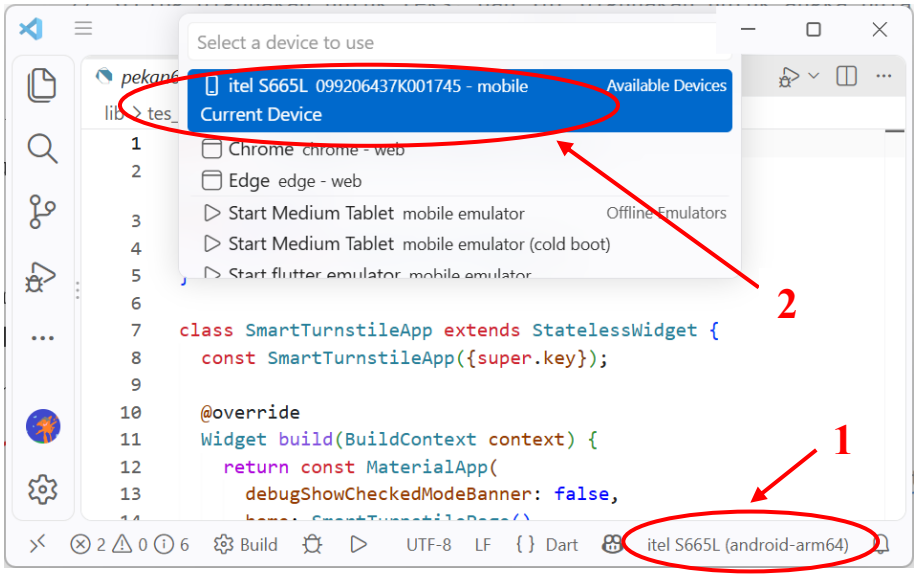
```dart
void main() {
  // TODO 1: Lengkapi tipe data yang tepat untuk teks dan angka bulat
  String nama = "Budi";
  int umur = 20;

  // TODO 2: Lengkapi perintah bawaan Dart untuk mencetak ke layar
  print("Halo, nama saya $nama dan umur saya $umur");

  // Kesimpulan pada soal ini adalah:
  // String digunakan untuk teks, dan int digunakan untuk angka bulat.
}
```

Perhatian

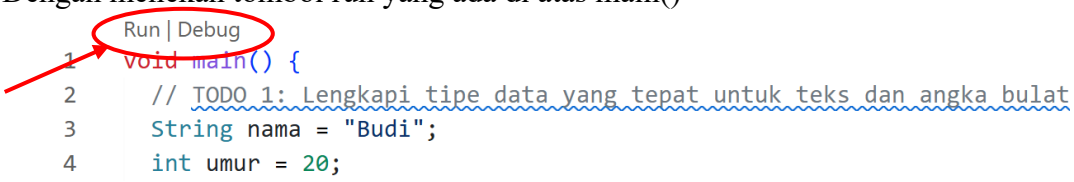
Sebelum menjalankan program, pastikan sudah memilih perangkat. Cara memilih perangkat dengan menekan tombol di pojok kanan bawah (1), kemudian pilih device (2).



Cara menjalankan program dart

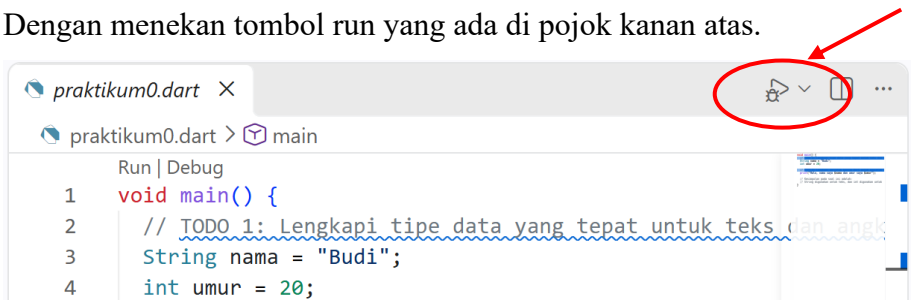
Cara Pertama

Dengan menekan tombol run yang ada di atas main()



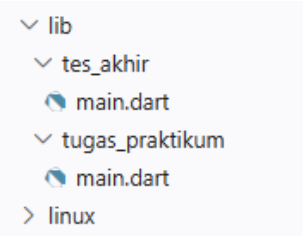
Cara Kedua

Dengan menekan tombol run yang ada di pojok kanan atas.



E.1. Aturan Penamaan Proyek
praktikum_pekan_07

E.2. Struktur Files



E.3. File Image
Buat struktur folder seperti berikut ini untuk menyimpan file gambar.
Praktikum_pekan_07
--- assets
    --- images
        --- terminal.png → gambar bebas

Aktifkan file assets dengan menambahkan baris code berikut ini pada file pubspec.yaml

```
59
60    # To add assets to your application, add an assets section, like this:
61    assets:
62      - assets/images/
63
```

E.4. Kerangka Dasar
Nama file: lib/tugas_praktikum/main.dart

Dinas Perhubungan Kota Bandung sedang menginisiasi proyek bernama Info Transit Kota Bandung. Sistem ini dirancang sebagai pusat kendali dan informasi publik bagi warga yang memanfaatkan moda transportasi perkotaan. Layar aplikasi menggabungkan empat modul informasi vital: identitas terminal transit terpadu, status kelancaran operasional harian, data proporsi ketersediaan armada transportasi, serta pemantauan visual fasilitas halte secara langsung.

Untuk membangun aplikasi ini, tim pengembang menerapkan pendekatan arsitektur modular, dimana setiap modul dibuat dengan function terpisah. Struktur utama layer dibentuk dengan menggunakan Column yang dibungkus SingleChildScrollView . Berikut adalah nama modul-modul tersebut:

1. buildHeaderCard() untuk kartu identitas terminal transit kota.
2. buildStatusBar() untuk status operasional dan kelancaran armada.
3. buildArmadaGrid() untuk sebaran armada Bus Utama dan Feeder pendukung.
4. buildFotoBannerStack() untuk banner pemantauan CCTV 24 jam.

Pada tahap persiapan ini, Anda menyusun kerangka dasar aplikasi (skeleton) pada berkas lib/tugas_praktikum/main.dart. Di awal pengerjaan, seluruh fungsi pembantu mengembalikan widget kosong SizedBox(). Hal ini menjamin program langsung dapat dikompilasi dan dijalankan (run) dengan sukses sejak awal tanpa memicu error, sebelum nantinya Anda melengkapi satu per satu logika antarmukanya pada Soal 1 hingga Soal 4:

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(MaterialApp(
    debugShowCheckedModeBanner: false,
    home: Scaffold(
      backgroundColor: Colors.grey[100],
      appBar: AppBar(
        title: Text('Info Transit Kota Bandung'),
        backgroundColor: Colors.blueGrey[900],
        foregroundColor: Colors.white,
      ),
      body: SingleChildScrollView(
        padding: EdgeInsets.all(16.0),
        child: Column(
          children: [
            buildHeaderCard(),
            SizedBox(height: 16),
            buildStatusBar(),
            SizedBox(height: 16),
            buildArmadaGrid(),
            SizedBox(height: 16),
            buildFotoBannerStack(),
          ],
        ),
      ),
    ),
  ));
}

// Soal 1
Widget buildHeaderCard() {
  return SizedBox();
}

// Soal 2
Widget buildStatusBar() {
  return SizedBox();
}

// Soal 3
Widget buildArmadaGrid() {
  return SizedBox();
}

// Soal 4
Widget buildFotoBannerStack() {
  return SizedBox();
}
```

E.5. Soal 1 - Kartu identitas terminal transit kota

Sebagai elemen identitas utama saat aplikasi pertama kali dibuka oleh warga, sistem memerlukan kartu identitas terminal yang di bagian atas layar bertuliskan "Terminal Kota Bandung" dengan sentuhan gradasi warna biru agar calon penumpang yakin bahwa mereka mengakses informasi resmi dari Pemkot Bandung.

```
// Soal 1
Widget buildHeaderCard() {
  return Container(
    width: double.infinity,
    // TODO 1: Atur padding kontainer sebesar 16 di semua sisi
    padding: EdgeInsets.all(_____),
    decoration: BoxDecoration(
      // TODO 2: Tentukan sudut membulat kontainer dengan radius 14
      borderRadius: BorderRadius.circular(_____),
      // TODO 3: Buat gradasi warna dari topLeft ke bottomRight
      //         dengan warna blue.shade900 dan blue.shade600
      gradient: LinearGradient(
        begin: Alignment._____,
        end: Alignment._____,
        colors: [_____, _____],
      ),
      // TODO 4: Terapkan bayangan BoxShadow dengan color black26,
      //         blurRadius 6, dan offset Y 3
      boxShadow: [
        BoxShadow(
          color: _____,
          blurRadius: _____,
          offset: Offset(0, _____),
        ),
      ],
    ),
    // TODO 5: Susun ikon bus dan teks judul terminal ke dalam
    //         baris Row yang sejajar di tengah (center)
    child: Row(
      crossAxisAlignment: CrossAxisAlignment._____,
      children: [
        Icon(Icons.directions_bus, color: Colors.white, size: 32),
        SizedBox(width: 12),
        Text('Terminal Kota Bandung', style: TextStyle(color: Colors.white, fontSize: 16)),
      ],
    ),
  );
}

```

E.6. Soal 2 - Status operasional dan kelancaran armada

Pengguna transportasi umum di Kota Bandung memerlukan kepastian jam layanan dan kelancaran arus armada sebelum bepergian. Dishub Kota Bandung ingin menyediakan status operasional ringkas yang menginformasikan jam kerja terminal (05.00 - 23.00 WIB) serta indikator kelancaran arus bus hari ini.

```dart
// Soal 2
Widget buildStatusBar() {
  return Container(
    padding: EdgeInsets.all(12),
    // TODO 6: Berikan dekorasi warna putih, borderRadius 8,
    //         dan border solid abu-abu (grey.shade300)
    decoration: BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.circular(_____),
      border: Border.all(color: _____),
    ),
    // TODO 7: Gunakan widget Row untuk menyusun informasi
    //         secara horizontal
    child: _____(
      children: [
        Icon(Icons.access_time, color: Colors.orange),
        SizedBox(width: 8),
        Text('Operasional: 05.00 - 23.00'),
        // TODO 8: Gunakan widget Spacer untuk mendorong
        //         badge status ke tepi kanan
        _____(),
        // TODO 9: Rancang Container badge dengan padding
        //         simetris (horizontal 8, vertical 4)
        //         dan borderRadius 6
        Container(
          padding: EdgeInsets.symmetric(horizontal: _____, vertical: _____),
          decoration: BoxDecoration(
            color: Colors.green.shade100,
            borderRadius: BorderRadius.circular(_____),
          ),
          // TODO 10: Tulis teks status 'LANCAR' dengan
          //          warna hijau tua (green.shade900)
          child: Text(
            '_____',
            style: TextStyle(color: _____),
          ),
        ),
      ],
    ),
  );
}
```

E.7. Soal 3 - Sebaran armada Bus Utama dan Feeder pendukung

Sistem transportasi terintegrasi Kota Bandung membagi beban armada menjadi dua kelompok utama, yaitu Bus Utama koridor primer (proporsi kapasitas 60%) dan angkutan Feeder pendukung (proporsi kapasitas 40%). Pusat kendali transportasi Kota Bandung perlu menampilkan distribusi armada ini secara berdampingan agar ketersediaan kendaraan terpantau seimbang dan transparan bagi warga.

```dart
// Soal 3
Widget buildArmadaGrid() {
  return Row(
    children: [
      // TODO 11: Bungkus Kartu Bus Utama dengan Expanded
      //          dan tentukan nilai flex 3 (proporsi 60%)
      _____(
        flex: _____,
        child: Container(
          padding: EdgeInsets.all(12),
          decoration: BoxDecoration(
            color: Colors.teal.shade50,
            borderRadius: BorderRadius.circular(8),
          ),
          child: Column(
            children: [
              // TODO 12: Atur Row di dalam kartu dengan
              //          MainAxisAlignment.spaceBetween
              Row(
                mainAxisAlignment: MainAxisAlignment._____,
                children: [
                  Icon(Icons.directions_bus, color: Colors.teal),
                  Text('60% KAPASITAS', style: TextStyle(fontSize: 9, color: Colors.teal)),
                ],
              ),
              SizedBox(height: 8),
              Text('18 Bus Utama', style: TextStyle(fontWeight: FontWeight.bold)),
            ],
          ),
        ),
      ),
      // TODO 13: Berikan pemisah horizontal antar kartu
      //          sebesar 12 pixel menggunakan SizedBox
      SizedBox(width: _____),
      // TODO 14: Bungkus Kartu LRT Feeder dengan Expanded
      //          dan tentukan nilai flex 2 (proporsi 40%)
      _____(
        flex: _____,
        child: Container(
          padding: EdgeInsets.all(12),
          decoration: BoxDecoration(
            color: Colors.indigo.shade50,
            borderRadius: BorderRadius.circular(8),
          ),
          child: Column(
            children: [
              // TODO 15: Atur Row di dalam kartu dengan
              //          MainAxisAlignment.spaceBetween
              Row(
```

```
                Row(
                  mainAxisAlignment: MainAxisAlignment._____,
                  children: [
                    Icon(Icons.tram, color: Colors.indigo),
                    Text('40%', style: TextStyle(fontSize: 9, color: Colors.indigo)),
                  ],
                ),
                SizedBox(height: 8),
                Text('6 LRT Feeder', style: TextStyle(fontWeight: FontWeight.bold)),
              ],
            ),
          ),
        ),
      ],
    );
  }

```

E.8. Soal 4 - Banner pemantauan CCTV 24 jam

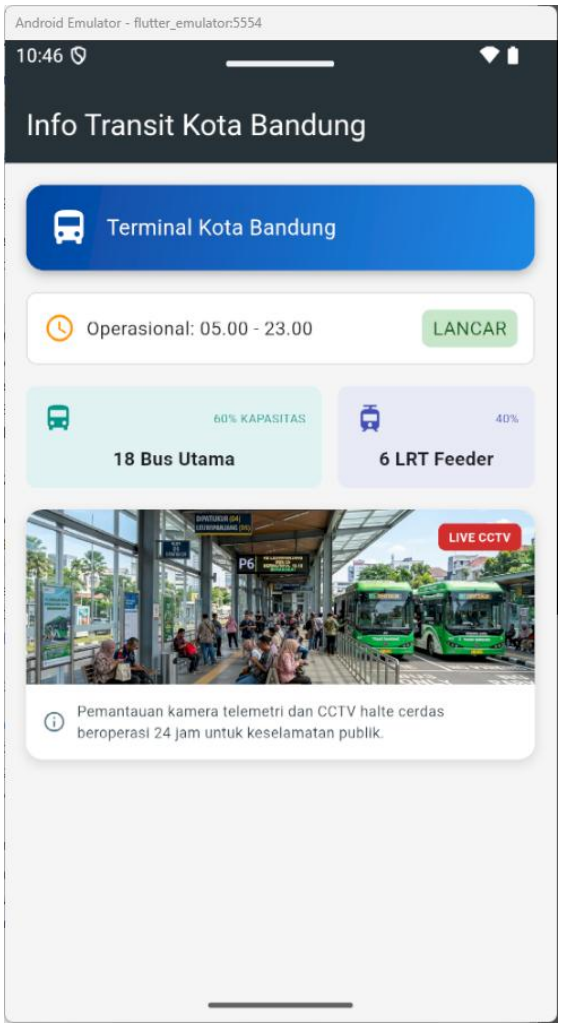
Menjamin keamanan warga saat menunggu angkutan umum di berbagai sudut Kota Bandung merupakan prioritas Dishub Kota Bandung. Fasilitas halte dan terminal dilengkapi pemantauan kamera pengawas selama 24 jam.

```dart
import 'package:flutter/material.dart';

// Soal 4
Widget buildFotoBannerStack() {
  // TODO 16: Inisialisasi Stack dengan clipBehavior: Clip.none
  //          agar elemen anak bisa menembus batas kartu
  return _____(
    clipBehavior: Clip._____,
    children: [
      Container(
        decoration: BoxDecoration(
          color: Colors.white,
          borderRadius: BorderRadius.circular(12),
          boxShadow: [
            BoxShadow(color: Colors.black12, blurRadius: 6, offset: Offset(0, 3)),
          ],
        ),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            // TODO 17: Bungkus Image.asset dengan ClipRRect dan
            //          atur borderRadius hanya pada bagian atas
            _____(
              borderRadius: BorderRadius.vertical(top: Radius.circular(_____)),
              // Keterangan Gambar:
              // Bebas (file apa saja berekstensi .png/.jpg di
              // assets/images/, tidak harus bertema terminal)
              // TODO 18: Tampilkan gambar lokal aset
              //          'assets/images/_____' dengan tinggi 130,
              //          lebar double.infinity, fit BoxFit.cover,
              //          dan sediakan errorBuilder
              child: Image.asset(
                'assets/images/_____',
                height: _____,
```

```
                height: _____,
                width: double.infinity,
                fit: BoxFit._____,
                errorBuilder: (context, error, stackTrace) {
                  return Container(
                    height: 130,
                    color: Colors.blueGrey[200],
                    child: Center(child: Icon(Icons.image, size: 40, color: Colors.blueGrey)),
                  );
                },
              ),
            ),
            Padding(
              padding: EdgeInsets.all(12),
              child: Row(
                children: [
                  Icon(Icons.info_outline, size: 18, color: Colors.blueGrey),
                  SizedBox(width: 8),
                  // TODO 19: Bungkus widget Text panjang dengan
                  //          Expanded agar tidak overflow
                  _____(
                    child: Text(
                      'Pemantauan kamera telemetri dan CCTV halte cerdas
                      beroperasi 24 jam untuk keselamatan publik.',
                      style: TextStyle(fontSize: 11, color: Colors.blueGrey[800]),
                    ),
                  ),
                ],
              ),
            ),
          ],
        ),
      ),
      // TODO 20: Posisikan badge mengambang pada koordinat
      //          top 10 dan right 10 dengan widget Positioned
      _____(
        top: _____,
        right: _____,
        child: Container(
          padding: EdgeInsets.symmetric(horizontal: 8, vertical: 4),
          decoration: BoxDecoration(
            color: Colors.red[700],
            borderRadius: BorderRadius.circular(6),
          ),
          child: Text(
            'LIVE CCTV',
            style: TextStyle(color: Colors.white, fontSize: 9, fontWeight: FontWeight.bold),
          ),
        ),
      ),
    ],
  );
}
```

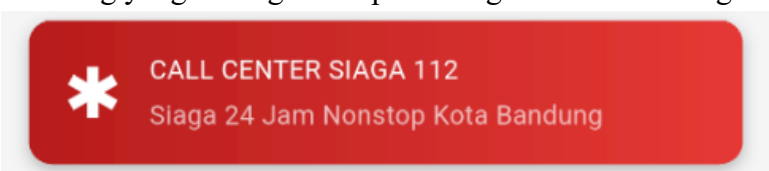
E.9. Hasil Akhir



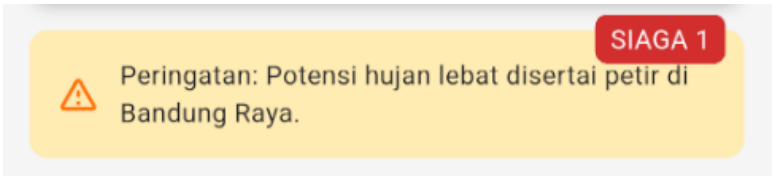
F. Tes Akhir

Pemerintah Kota Bandung sedang merancang prototipe aplikasi mobile tanggap darurat warga Bandung Emergency 112 dengan ketentuan:

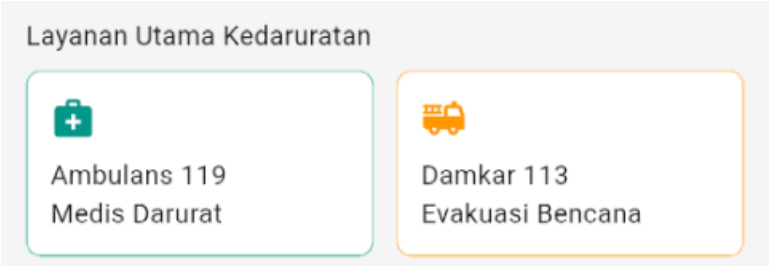
1. Header Aplikasi (Identitas Resmi Layanan Darurat):
   - Menggunakan Container lebar penuh (double.infinity), sudut membulat 16px, bayangan lembut BoxShadow, dan gradasi warna LinearGradient merah resmi (Colors.red.shade900 ke Colors.red.shade600).
   - Di dalamnya tersusun baris Row yang memuat ikon darurat (Icons.emergency), judul "CALL CENTER SIAGA 112", serta teks jaminan siaga 24 jam untuk warga Kota Bandung yang dibungkus Expanded agar aman di berbagai ukuran layar.



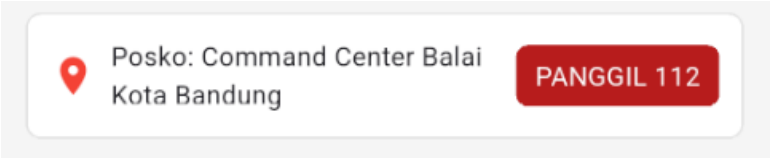
2. Banner Status Kota (Penerapan Stack & Positioned):
    - Menggunakan Stack dengan clipBehavior: Clip.none untuk menempatkan lencana status darurat mengambang di pojok kanan atas banner (Positioned(top: -10, right: 12, ...)).
    - Isi banner memuat informasi singkat peringatan cuaca ekstrem BMKG Bandung Raya dengan ikon peringatan dan teks terstruktur.



3. Menu Cepat 2 Layanan Utama (1 Baris x 2 Kolom Berbobot Seimbang):
    - Mengombinasikan Row dan dua buah widget Expanded untuk menampilkan 2 kartu layanan darurat utama yang membagi lebar layar secara proporsional:
        - Ambulans 119 (Ikon: Icons.medical_services, Aksen: Hijau/Teal).
        - Damkar 113 (Ikon: Icons.fire_truck, Aksen: Oranye).
    - Setiap kartu memiliki padding dalam, sudut membulat, garis batas tepi tipis (border), bayangan halus, serta teks judul dan deskripsi yang rapi.



4. Kontak Posko Komando Cepat (Anti-Overflow Footer Card):
    - Berisi baris Row dengan ikon penanda lokasi (Icons.location_on), teks alamat posko siaga di Balai Kota Bandung yang dibungkus Expanded (mencegah RenderFlex overflow), serta tombol aksi cepat "PANGGIL 112" dengan warna merah mencolok.



G. Jurnal Praktikum

Untuk pengerjaan Tes Akhir di atas, Anda diwajibkan menyusun Jurnal Praktikum dalam format dokumen (pdf atau word) dengan struktur:

1. Halaman Sampul: Nama, NIM, Kelas, dan Judul Praktikum.
2. Pembahasan dan Analisis Program:
   Jelaskan alur kerja program yang Anda buat. Anda dapat menyertakan potongan kode (snippet) pada bagian-bagian penting. Lampirkan screenshot dari output dan berikan analisis singkat mengenai hasil output tersebut
3. Kesimpulan Praktikum
   Tuliskan kesimpulan menyeluruh mengenai apa yang telah Anda pelajari pada sesi praktikum hari ini.

H. Cara Pengumpulan

1. Buatlah sebuah folder baru di komputer Anda dan beri nama dengan format: pekan7_NIM_NamaLengkap (Contoh: pekan7_120220001_BudiSantoso).
2. Masukkan proyek flutter dan seluruh file program Praktikum serta Tugas Akhir ke dalam folder tersebut.
3. Sebelum memasukkan proyek flutter ke folder, jalankan perintah **flutter clean** pada terminal.
4. Masukkan juga file Jurnal Praktikum ke folder tersebut.
5. Compress atau arsipkan folder yang sudah berisi file-file tersebut menjadi satu file berformat .zip atau .rar.
6. Upload ke LMS.